RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE LA FORMATION ET DE L'ENSEIGNEMENT PROFESSIONNELS

Centre de formation professionnel et d'Apprentissage
Des Arts Traditionnels

Spécialité : INT 0703 — Technicien Supérieur en Informatique / Bases De Données

MÉMOIRE DE FIN D'ÉTUDES

Pour l'obtention du Brevet de Technicien Supérieur (BTS)

THÈME DU PROJET :

CONCEPTION ET RÉALISATION D'UN SYSTÈME DE GESTION INTÉGRÉE DES RÉSIDENCES UNIVERSITAIRES (RÉSIVO)

Modélisation de Base de Données Relationnelle, Affectation Automatisée par Pavillon et Genre, Traitement des Doléances Techniques et Synchronisation Temps Réel Multi-Surfaces

Organisme d'accueil : Direction des Œuvres Universitaires (DOU) — Cité Universitaire (Tlemcen)

Présenté par l'étudiant :

• Mohamed Anes DEV

Stagiaire Technicien Supérieur

Option : Bases de Données

Sous la direction de l'encadrant :

• Enseignant Encadreur Pédagogique

• Maître de Stage Professionnel (DOU)

Session / Promotion : 2026

DÉDICACES

Ce travail scientifique et technique est dédié avec une profonde gratitude :

À des parents dévoués, dont les sacrifices inlassables, le soutien moral constant et la confiance indéfectible ont constitué le moteur indispensable à l'accomplissement de ces années de formation professionnelle supérieure.

Aux enseignants et formateurs du Centre de formation professionnelle et d'apprentissage des arts traditionnels, pour la qualité de leur transmission pédagogique, leur rigueur méthodologique et leurs conseils éclairés en modélisation de bases de données et en ingénierie logicielle.

À l'ensemble des camarades de promotion de la spécialité Informatique / Bases de Données (INT 0703), en souvenir des séances de travail collaboratif, des échanges techniques stimulants et des défis académiques relevés ensemble.

À toute personne ayant contribué de près ou de loin à l'aboutissement de ce projet.

REMERCIEMENTS

Au terme de ce projet de fin d'études, il convient d'exprimer des remerciements sincères envers toutes les personnes qui ont permis la réalisation et la concrétisation de ce travail.

Une reconnaissance particulière est adressée aux membres de la Direction des Œuvres Universitaires (DOU) de Tlemcen, pour leur accueil favorable au sein de leurs structures, leur disponibilité et les précieuses données de terrain communiquées, permettant une compréhension exhaustive des défis opérationnels liés à l'hébergement étudiant.

Des remerciements appuyés sont également formulés à l'égard de l'équipe pédagogique du Centre pour l'encadrement méthodologique et technique assuré tout au long du cycle de formation INT 0703.

Enfin, un hommage respectueux est rendu aux membres du jury qui ont accepté d'examiner et d'évaluer ce mémoire avec bienveillance et rigueur scientifique.

TABLE DES MATIÈRES

DÉDICACES i

REMERCIEMENTS ii

TABLE DES MATIÈRES iii

LISTE DES TABLEAUX ET DES FIGURES iv

INTRODUCTION GÉNÉRALE 1

CHAPITRE I : CONTEXTE GÉNÉRAL, ÉTAT DE L'ART ET PROBLÉMATIQUE 3

I.1 Présentation de l'Organisme d'Accueil (Direction des Œuvres Universitaires) 3

I.2 Analyse du Processus Traditionnel de Gestion des Résidences 4

I.3 Diagnostic Critique de l'Existant et Dysfonctionnements Constatés 5

I.4 Formulation de la Question Centrale Unique de Recherche 6

I.5 Objectifs Opérationnels du Projet RÉSIVO 7

CHAPITRE II : MÉTHODOLOGIE ET ANALYSE DES BESOINS 8

II.1 Démarche Méthodologique (Cycle de Vie et Approche Structurée) 8

II.2 Identification des Acteurs et Rôles Système 9

II.3 Règles de Gestion Fondamentales (RG-01 à RG-10) 10

II.4 Spécifications Fonctionnelles Détaillées 11

II.5 Spécifications Non Fonctionnelles et Contraintes d'Exploitation 12

CHAPITRE III : CONCEPTION ET MODÉLISATION DE LA BASE DE DONNÉES 13

III.1 Démarche MERISE et Cycle d'Abstraction des Données 13

III.2 Modèle Conceptuel des Données (MCD) et Cardinalités 14

III.3 Dictionnaire des Données Exhaustif 15

III.4 Modèle Logique des Données Relationnel (MLD-R) et Normalisation 3FN 17

III.5 Modèle Physique des Données (MPD) PostgreSQL / Supabase 18

III.6 Triggers et Fonctions Stockées d'Intégrité Métier (PL/pgSQL) 19

III.7 Sécurité Avancée et Contrôle d'Accès par Ligne (Row Level Security - RLS) 21

CHAPITRE IV : RÉALISATION ET IMPLÉMENTATION DU SYSTÈME 23

IV.1 Architecture Logicielle Globale Multi-Surfaces 23

IV.2 Environnement Technique et Outillage Logiciel 24

IV.3 Développement de la Console Desktop d'Administration (Python / PySide6) 25

IV.4 Développement du Portail Web Étudiant (Astro / TypeScript / Tailwind) 27

IV.5 Protocole d'Intégration MCP (Model Context Protocol) et Piste d'Audit 28

IV.6 Packaging, Compilation et Distribution (Installeur Windows Inno Setup v0.0.1) 29

CHAPITRE V : TESTS, SÉCURITÉ ET VALIDATION DES RÉSULTATS 30

V.1 Stratégie Globale de Test et Démarche de Validation 30

V.2 Résultats des Tests Unitaires et d'Intégration (Pytest - 29 tests validés) 31

V.3 Validation des Contraintes d'Intégrité de Base de Données 32

V.4 Tests du Portail Web et Conformité Client (Vitest - 10 tests validés) 33

V.5 Tests de Non-Régression du Serveur MCP (97 tests validés) 34

CHAPITRE VI : DISCUSSION ET PERSPECTIVES 35

VI.1 Confrontation des Résultats à la Question Centrale de Recherche 35

VI.2 Analyse Comparative avec les Systèmes Existants 36

VI.3 Limites Identifiées et Contraintes Opérationnelles 37

VI.4 Perspectives d'Évolution et Pistes Futures 38

CONCLUSION GÉNÉRALE 39

BIBLIOGRAPHIE TRIPARTITE 41

1. Les Ouvrages 41

2. Les Articles et Publications Scientifiques 41

3. Les Sites Internet et Documentations Officielles 42

ANNEXES I

RÉSUMÉ ET MOTS-CLÉS (DOS DU MÉMOIRE) 43

LISTE DES TABLEAUX ET DES FIGURES

A. Liste des Tableaux

Tableau 1.1 : Matrice comparative des dysfonctionnements du système manuel vs RÉSIVO ..... Page 5

Tableau 2.1 : Matrice des acteurs et privilèges système ..... Page 9

Tableau 2.2 : Catalogue des règles de gestion d'intégrité (RG-01 à RG-10) ..... Page 10

Tableau 3.1 : Dictionnaire des données — Entités BLOCS et ROOMS ..... Page 15

Tableau 3.2 : Dictionnaire des données — Entité STUDENTS ..... Page 16

Tableau 3.3 : Dictionnaire des données — Entités MAINTENANCE et ROOM_CHANGES ..... Page 16

Tableau 3.4 : Synthèse des politiques Row Level Security (RLS) ..... Page 22

Tableau 5.1 : Récapitulatif d'exécution des suites de tests automatisés (136 tests validés) ..... Page 31

B. Liste des Figures et Schémas

Figure 1.1 : Cartographie organisationnelle des flux d'admission manuels en cité universitaire ..... Page 4

Figure 2.1 : Diagramme synthétique des cas d'utilisation (Use Case) ..... Page 11

Figure 3.1 : Modèle Conceptuel des Données (MCD - Formalisme Entité/Association) ..... Page 14

Figure 3.2 : Schéma Relationnel et Contraintes de Clés Étrangères (MPD) ..... Page 18

Figure 3.3 : Arbre d'exécution du trigger automatique d'héritage de genre ..... Page 20

Figure 4.1 : Architecture distribuée globale (Desktop, Web, Base Cloud Supabase, MCP) ..... Page 23

Figure 4.2 : Capture de la console d'administration Desktop (Gestion des Blocs et Chambres) ..... Page 26

Figure 4.3 : Capture du formulaire de création de chambre avec héritage automatique de genre ..... Page 27

Figure 4.4 : Capture du portail Web étudiant (Tableau de bord et doléances techniques) ..... Page 28

INTRODUCTION GÉNÉRALE

La gestion des infrastructures d'hébergement au sein des campus universitaires constitue l'un des piliers fondamentaux des œuvres universitaires. Chaque rentrée académique confronte les administrations des résidences à un flux massif d'étudiants sollicitant une chambre, requérant une organisation logistique extrêmement rigoureuse, une équité stricte dans l'attribution des places et une gestion fluide des doléances d'entretien quotidiennes.

Traditionnellement, ces opérations demeurent largement tributaires de registres sur support papier, de classeurs physiques ou de feuilles de calcul disparates non synchronisées. Cette organisation manuelle engendre des risques majeurs : incohérences dans le suivi des capacités résiduelles, erreurs humaines dans l'attribution des dortoirs, absence d'étanchéité entre pavillons masculins et féminins, et lenteur excessive dans la prise en charge des pannes signalées par les étudiants résidents.

■ QUESTION CENTRALE UNIQUE DE RECHERCHE

« Comment concevoir et déployer une architecture de base de données relationnelle sécurisée et distribuée capable d'automatiser l'affectation des étudiants en chambres selon des contraintes strictes d'intégrité (genre par pavillon, capacité résiduelle, vérification du statut d'admission) tout en offrant une synchronisation bidirectionnelle en temps réel entre le portail étudiant et la console administrative ? »

Pour apporter une réponse concrète et scientifiquement éprouvée à cette interrogation fondamentale, le présent projet de fin d'études a consisté en l'ingénierie et le développement complet de la plateforme RÉSIVO (Système de Gestion Intégrée des Résidences Universitaires). L'architecture élaborée repose sur un moteur de base de données relationnelle PostgreSQL hébergé sur Supabase, couplé à une console d'administration Desktop performante développée en Python/PySide6 pour le personnel de gestion, un portail Web moderne en Astro pour les étudiants résidents, et une interface programmatique standardisée via le protocole MCP.

La démarche d'investigation et de conception s'est articulée autour de la méthode MERISE, accordant une priorité absolue à l'intégrité des données au niveau du moteur serveur grâce à des triggers PL/pgSQL automatisés, des procédures stockées transactionnelles et des politiques de filtrage par ligne (Row Level Security).

Le présent document est structuré en six chapitres distincts :

• Le Chapitre I expose le contexte général, l'analyse de l'organisme d'accueil, le diagnostic de l'existant et les objectifs visés ;

• Le Chapitre II détaille la démarche méthodologique, l'inventaire des acteurs et les règles de gestion du domaine ;

• Le Chapitre III approfondit la modélisation conceptuelle, logique et physique de la base de données, ainsi que les mécanismes d'intégrité serveur ;

• Le Chapitre IV décrit la réalisation technique des différentes couches applicatives (Desktop, Web, Déploiement) ;

• Le Chapitre V présente la méthodologie de test, la validation des résultats et les métriques de fiabilité ;

• Le Chapitre VI ouvre une discussion approfondie sur la portée des résultats obtenus, les limites rencontrées et les perspectives d'évolution.

CHAPITRE I : CONTEXTE GÉNÉRAL, ÉTAT DE L'ART ET PROBLÉMATIQUE

I.1 Présentation de l'Organisme d'Accueil

L'étude a été menée en étroite liaison avec la Direction des Œuvres Universitaires (DOU), établissement public sous tutelle ministérielle chargé de la mise en œuvre des prestations sociales au bénéfice de la communauté estudiantine. Parmi ses prérogatives majeures figurent l'hébergement en cité universitaire, la restauration, le transport et les activités culturelles et sportives.

La cité universitaire de Tlemcen, sélectionnée comme cadre expérimental, regroupe plusieurs milliers de lits répartis dans des pavillons distincts (communément dénommés « Blocs »). Chaque bloc comprend plusieurs étages, des chambres individuelles ou collectives (de deux à trois lits), des espaces sanitaires et des locaux communs.

I.2 Analyse du Processus Traditionnel de Gestion des Résidences

L'audit du circuit administratif actuel met en lumière un processus en plusieurs étapes chronologiques :

1. Dépôt physique ou semi-numérique du dossier d'admission par l'étudiant bachelier ou réinscrit ;
2. Contrôle de conformité de l'inscription pédagogique auprès des facultés de rattachement ;
3. Attribution manuelle d'une chambre par les agents de scolarité au moyen de registres manuscrits ;
4. Remise des clés et installation matérielle du résident ;
5. Déclaration des pannes d'équipements sur cahier de doléances déposé au bureau du régisseur de pavillon.

I.3 Diagnostic Critique de l'Existant et Dysfonctionnements Constatés

L'analyse rigoureuse de ce mode opératoire conventionnel révèle de profondes faiblesses structurelles qui nuisent à l'efficacité du service public universitaire :

• Risque élevé d'erreurs d'attribution : L'absence de vérification automatique permet la sur-occupation de chambres ou l'affectation involontaire d'étudiants dans un pavillon non conforme à leur genre légal ;
• Désynchronisation des informations : Les modifications effectuées par un service (ex. changement de chambre ou radiation) ne sont pas répercutées en temps réel sur les fiches de surveillance ni sur le service de maintenance ;
• Traçabilité déficiente des pannes : Les demandes d'intervention technique écrites sur registre physique sont régulièrement égarées, sans horodatage objectif ni suivi d'état (en attente, en cours, résolu) ;
• Manque de transparence pour l'étudiant : Le résident est privé de visibilité sur l'avancement de son dossier d'admission ou le traitement de sa requête technique.

Tableau 1.1 : Matrice comparative des dysfonctionnements du système manuel vs la solution RÉSIVO

Critère d'Évaluation	Gestion Manuelle / Fichiers Disparates	Solution Numérique RÉSIVO
Attribution des chambres	Manuelle, sujette à surcapacité et erreurs	Automatisée, contrainte par triggers SQL
Ségrégation par genre	Vérification visuelle humaine aléatoire	Héritage strict et inviolable au niveau bloc
Délai de traitement doléances	Plusieurs jours à semaines, registres perdus	Horodatage temps réel avec suivi photographique
Vérification scolarité	Vérification sur listing papier fastidieuse	Import CSV officiel et réconciliation automatique

Sécurité & Traçabilité	Aucun journal d'accès, registre falsifiable	Chiffrement TLS, RLS PostgreSQL et piste d'audit MCP

I.4 Formulation de la Question Centrale Unique de Recherche

Au regard des insuffisances méthodologiques observées, la recherche doctorale et appliquée s'est cristallisée autour d'une question directrice unique (Slide 6 du document cadre) :

> ■ **ÉNONCÉ FORMEL DE LA PROBLÉMATIQUE**
> « Par quels mécanismes conceptuels et algorithmiques une base de données relationnelle moderne peut-elle garantir l'inviolabilité des contraintes d'affectation spatiale et de genre en cité universitaire, tout en permettant une administration ergonomique locale et un accès ubiquitaire pour les étudiants résidents ? »

I.5 Objectifs Opérationnels du Projet RÉSIVO

Pour apporter une solution définitive à cette problématique, les objectifs suivants ont été fixés :

1. Modéliser une base de données relationnelle en 3e Forme Normale assurant l'intégrité référentielle complète ;
2. Implémenter la logique métier d'affectation par genre au niveau même de la base (Triggers PL/pgSQL) pour éliminer tout risque de contournement applicatif ;
3. Concevoir une application de bureau native destinée aux administrateurs de la cité universitaire ;
4. Déployer un portail Web réactif et accessible destiné aux étudiants résidents ;
5. Intégrer un système de reporting technique photographique pour les interventions de maintenance ;
6. Valider l'ensemble du dispositif par des protocoles d'essais unitaires et fonctionnels automatisés.

CHAPITRE II : MÉTHODOLOGIE ET ANALYSE DES BESOINS

II.1 Démarche Méthodologique

La méthodologie adoptée s'inscrit dans le cadre de la méthode d'analyse et de conception MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise), particulièrement adaptée aux systèmes d'information fortement centrés sur les bases de données relationnelles, complétée par une démarche agile pour le prototypage des interfaces utilisateur.

La démarche a suivi trois cycles successifs :
• Cycle d'abstraction des données : Passage rigoureux du niveau conceptuel (MCD) au niveau logique (MLD) puis physique (MPD) ;
• Cycle de traitement : Définition des règles de gestion, des cas d'utilisation et des procédures transactionnelles stockées ;
• Cycle de réalisation et validation : Développement itératif, rédaction de tests unitaires et intégration continue.

II.2 Identification des Acteurs et Rôles Système

Le système d'information RÉSIVO distingue quatre catégories principales d'acteurs interagissant avec la base de données :

Tableau 2.1 : Matrice des acteurs et privilèges système

Acteur	Nature d'Accès	Prérogatives et Périmètre Fonctionnel
Étudiant Résident	Portail Web distant	Soumission de candidature, consultation d'affectation, signalement de panne avec photo, demande de mutation de chambre.
Agent Administratif	Console Desktop locale	Examen et validation des

		candidatures, gestion des pavillons et chambres, attribution manuelle ou assistée, importation des listes officielles.
Superviseur / Direction	Console Desktop locale	Gestion des comptes du personnel, supervision de l'état d'occupation global, suppression et audit des actions système.
Agent de Maintenance	Console & Reporting	Réception des tickets d'intervention, mise à jour des statuts (en cours, résolu), consultation des notes techniques.

II.3 Règles de Gestion Fondamentales

Afin d'éliminer toute ambiguïté lors de la modélisation des données, un catalogue strict de règles de gestion a été formalisé :

Tableau 2.2 : Catalogue des règles de gestion d'intégrité (RG-01 à RG-10)

Code Règle	Intitulé	Énoncé Formel de la Règle
RG-01	Désignation de Genre de Pavillon	Chaque pavillon (Bloc) possède obligatoirement un genre désigné immuable (Masculin ou Féminin).
RG-02	Héritage Automatique de Genre	Toute chambre créée au sein d'un bloc hérite automatiquement et exclusivement du genre désigné de son bloc de

			rattachement.
	RG-03	Conformité de Genre Résident	Un étudiant ne peut être affecté que dans une chambre dont le genre correspond strictement à son genre légal d'état-civil.
	RG-04	Capacité Résiduelle Inviolable	Le nombre d'étudiants actifs affectés à une chambre ne peut excéder la capacité maximale autorisée (de 1 à 8 lits).
	RG-05	Verrouillage de Capacité Occupée	La capacité d'une chambre ne peut être diminuée en deçà du nombre actuel de ses occupants réguliers.
	RG-06	Verrouillage de Suppression	Une chambre ou un bloc hébergeant des résidents actifs ne peut en aucun cas être supprimé de la base de données.
	RG-07	Unicité de Numérotation Spatiale	Le couple (Bloc, Numéro de chambre) constitue une clé unique absolue au sein de la résidence.
	RG-08	Vérification Scolarité Obligatoire	Seuls les étudiants figurant sur la liste officielle d'inscription universitaire peuvent voir leur candidature validée.
	RG-09	Traçabilité des Doléances	Toute doléance de réparation est horodatée, rattachée à l'étudiant émetteur et classée

		selon une catégorie technique normative.
RG-10	Mutation Régulée	Une demande de changement de chambre requiert un motif explicite et une chambre cible respectant la stricte conformité de genre.

II.4 Spécifications Fonctionnelles Détaillées

Le système répond aux cas d'utilisation primordiaux suivants :
• Module Blocs & Chambres : Création préalable des pavillons avec attribution de genre, suivi en temps réel de la disponibilité des lits par jauge graphique, modification et archivage sécurisé ;
• Module Candidatures & Affectation : Filtrage instantané des étudiants en attente, consultation de la fiche signalétique avec photo, sélection automatique des chambres compatibles non complètes, validation transactionnelle ;
• Module Liste Officielle : Importation de fichiers CSV normalisés contenant les inscrits universitaires et détection immédiate des écarts d'identité ;
• Module Maintenance : Soumission de doléance avec téléversement de cliché photographique probant, prise en charge par les agents techniques et clôture motivée ;
• Module Mutations : Comparaison entre chambre actuelle et chambre souhaitée, vérification des places vacantes et transfert atomique en base de données.

II.5 Spécifications Non Fonctionnelles

• Sécurité et Confidentialité : Chiffrement des communications en transit (HTTPS / WSS), hachage des mots de passe selon l'algorithme Argon2/Bcrypt et isolement des enregistrements via Row Level Security (RLS) ;
• Performance et Réactivité : Temps d'exécution des requêtes de recherche inférieur à 100 millisecondes, indexation des colonnes de filtrage fréqent (block,

gender, status) ;

• Portabilité et Ergonomie : Fonctionnement autonome sur postes de travail Windows sans dépendance logicielle complexe, et compatibilité multi-navigateurs pour le portail Web.

CHAPITRE III : CONCEPTION ET MODÉLISATION DE LA BASE DE DONNÉES

III.1 Démarche MERISE et Cycle d'Abstraction

En adéquation avec les compétences cibles de la formation INT 0703 (Technicien Supérieur en Informatique / Bases de Données), la conception du référentiel de données a constitué le noyau technique du projet. La démarche a respecté les principes de la modélisation conceptuelle, en identifiant rigoureusement les entités du domaine, leurs attributs élémentaires et les associations sémantiques qui les relient.

III.2 Modèle Conceptuel des Données (MCD)

Le Modèle Conceptuel des Données formalise les règles de gestion sous forme d'entités-types et d'associations :

• Entité BLOC : Représente les pavillons de la résidence. Identifiée par son nom unique (A, B, C...) et caractérisée par son genre désigné ;

• Entité ROOM : Représente chaque unité de logement. Rattachée à un BLOC, caractérisée par son étage, son numéro, sa capacité et son genre ;

• Entité STUDENT : Représente l'étudiant postulant ou hébergé. Reliée à la chambre par une association d'affectation (0,1 à 1,n) ;

• Entité STAFF : Représente les agents et administrateurs habilités ;

• Entité MAINTENANCE_REPORT : Représente les doléances techniques soumises par les résidents ;

• Entité ROOM_CHANGE_REQUEST : Représente les demandes formelles de transfert de dortoir ;

• Entité OFFICIAL_RESIDENT : Représente le registre officiel universitaire de contrôle.

III.3 Dictionnaire des Données Exhaustif

Le dictionnaire des données spécifie la nature, le type et les contraintes associées à chaque champ manipulé par le système d'information :

Tableau 3.1 : Dictionnaire des données — Entités BLOCS et ROOMS

Attribut	Description	Type SQL	Contraintes
blocks.id	Identifiant unique du pavillon	UUID	Clé Primaire, NOT NULL
blocks.name	Désignation alphabétique du bloc	TEXT	UNIQUE, NOT NULL
blocks.gender	Genre désigné du bloc	gender (ENUM)	'male' | 'female', NOT NULL
blocks.created_at	Horodatage de création	TIMESTAMPTZ	DEFAULT now(), NOT NULL
rooms.id	Identifiant unique de la chambre	UUID	Clé Primaire, DEFAULT gen_random_uuid()
rooms.block	Référence textuelle au pavillon	TEXT	NOT NULL, REFERENCES blocks(name)
rooms.floor	Étage d'implantation du local	INTEGER	DEFAULT 0, CHECK (floor >= 0)
rooms.number	Numéro de porte de la chambre	TEXT	NOT NULL, UNIQUE(block, number)
rooms.capacity	Nombre total de lits	INTEGER	DEFAULT 1, CHECK (capacity > 0)

rooms.gender	Genre imposé à la chambre	gender (ENUM)	Synchronisé automatiquement par trigger

Tableau 3.2 : Dictionnaire des données — Entité STUDENTS

Attribut	Description	Type SQL	Contraintes
students.id	Identifiant profil de l'étudiant	UUID	Clé Primaire, REFERENCES profiles(id) ON DELETE CASCADE
students.card_id	Numéro de carte d'étudiant / Bac	TEXT	UNIQUE, NOT NULL
students.first_name	Prénom de l'étudiant	TEXT	NOT NULL
students.last_name	Nom de famille de l'étudiant	TEXT	NOT NULL
students.birth_date	Date de naissance légale	DATE	NOT NULL
students.gender	Genre de l'étudiant	gender (ENUM)	'male' | 'female', NOT NULL
students.phone	Numéro de téléphone mobile	TEXT	NOT NULL
students.study_level	Cycle d'études supérieures	study_level (ENUM)	'L1','L2','L3','M1','M2','Doctorat'
students.status	Statut de la demande d'asile	student_status (ENUM)	'pending','active','rejected', DEFAULT 'pending'
students.room_id	Chambre actuellement	UUID	NULLABLE, REFERENCES rooms(id) ON DELETE SET

assignée	NULL

III.4 Modèle Logique des Données Relationnel (MLD-R)

La dérivation du MCD vers le Modèle Logique Relationnel s'est effectuée dans le respect strict des formes normales jusqu'à la 3e Forme Normale (3FN). Chaque attribut non clé dépend directement et pleinement de la clé primaire entière, éliminant toute dépendance transitive et prévenant les anomalies de mise à jour.

Structure relationnelle textuelle :

• BLOCKS (id [PK], name [UNIQUE], gender, description, created_at, updated_at)

• ROOMS (id [PK], block, floor, number, capacity, gender, created_at, updated_at) [UNIQUE(block, number)]

• STUDENTS (id [PK], card_id [UNIQUE], first_name, last_name, birth_date, gender, phone, email, university, faculty, study_level, wilaya, status, rejection_reason, room_id [FK], reviewed_by [FK], reviewed_at, created_at, updated_at)

• STAFF (id [PK], full_name, position, created_at)

• MAINTENANCE_REPORTS (id [PK], student_id [FK], category, description, photo_path, status, staff_note, handled_by [FK], created_at, updated_at)

• ROOM_CHANGE_REQUESTS (id [PK], student_id [FK], current_room_id [FK], preferred_room_id [FK], reason, status, decision_note, decided_by [FK], decided_at, created_at, updated_at)

• OFFICIAL_RESIDENTS (id [PK], card_id [UNIQUE], first_name, last_name, birth_date, gender, university, faculty, created_at)

III.5 Triggers et Procédures Stockées d'Intégrité Métier (PL/pgSQL)

Une contribution majeure de ce travail de fin d'études a consisté à ancrer les contrôles de sécurité et de consistance directement dans le moteur de base de

données PostgreSQL via le langage procédural PL/pgSQL. Cette démarche garantit que même en cas d'accès direct par script ou par API tierce, aucune donnée corrompue ne peut être enregistrée.

■ CODE DU TRIGGER D'HÉRITAGE AUTOMATIQUE DU GENRE DE BLOC

```
CREATE OR REPLACE FUNCTION sync_room_gender_from_block()
RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE b_gender gender;
BEGIN
  SELECT gender INTO b_gender FROM blocks WHERE name = NEW.block;
  IF b_gender IS NOT NULL THEN
    NEW.gender := b_gender;
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trigger_sync_room_gender
BEFORE INSERT OR UPDATE ON rooms
FOR EACH ROW EXECUTE FUNCTION sync_room_gender_from_block();
```

Le déclencheur ci-dessus intercepte chaque opération d'insertion ou de mise à jour sur la table des chambres. Il recherche le genre légal assigné au pavillon parent et l'affecte de façon transparente à la chambre. Cette implémentation matérialise l'exigence fonctionnelle demandée : l'utilisateur choisit le bloc, et la chambre reçoit automatiquement son genre sans risque d'erreur humaine.

■ CODE DU TRIGGER DE VERROUILLAGE D'OCCUPATION (ROOMS_GUARD)

```
CREATE OR REPLACE FUNCTION rooms_guard()
RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE occ integer;
BEGIN
  IF TG_OP = 'DELETE' THEN
    SELECT count(*) INTO occ FROM students WHERE room_id = OLD.id AND status
= 'active';
    IF occ > 0 THEN
      RAISE EXCEPTION 'Suppression interdite : la chambre héberge % résident(s)
```

```
actif(s)',                                                                    occ;
   END                                                                        IF;
   RETURN                                                                     OLD;
 ELSIF TG_OP = 'UPDATE' THEN
   SELECT count(*) INTO occ FROM students WHERE room_id = NEW.id AND status
=                                                                             'active';
   IF NEW.capacity < occ THEN
    RAISE EXCEPTION 'Capacité invalide : la capacité (%) ne peut être inférieure aux
résidents (%)', NEW.capacity, occ;
   END                                                                        IF;
   IF occ > 0 AND NEW.gender <> OLD.gender THEN
    RAISE EXCEPTION 'Modification de genre interdite : la chambre compte des
résidents                                                                     actifs';
   END                                                                        IF;
   RETURN                                                                     NEW;
 END                                                                          IF;
END;
$$;
```

III.6 Sécurité Avancée : Row Level Security (RLS)

Pour respecter les impératifs de confidentialité des données à caractère personnel (RGPD et législation nationale), le moteur PostgreSQL a été verrouillé par des politiques RLS (Row Level Security). Contrairement à un modèle d'habilitation applicatif traditionnel, la base de données évalue l'identité du demandeur (auth.uid()) pour chaque tuple renvoyé.

Tableau 3.4 : Synthèse des politiques Row Level Security (RLS)

Table Protégée	Opération	Condition d'Autorisation (Policy USING / WITH CHECK)
blocks	SELECT	auth.role() = 'authenticated' (Lecture ouverte à tout utilisateur connecté)
blocks	INSERT, UPDATE, DELETE	is_staff() = true (Gestion

		réservée au personnel administratif)
rooms	SELECT	auth.role() = 'authenticated'
rooms	INSERT, UPDATE, DELETE	is_staff() = true (Édition sous contrôle du personnel)
students	SELECT (Lecture)	id = auth.uid() OR is_staff() (Étudiant consulte son dossier, personnel consulte tous)
students	UPDATE	is_staff() = true (Seul le personnel valide les admissions)
maintenance_reports	INSERT	student_id = auth.uid() AND status = 'pending' (Étudiant déclare ses propres pannes)
maintenance_reports	ALL	is_staff() = true (Le personnel instruit et clôture les doléances)
official_residents	ALL	auth_role() = 'admin' (Table universitaire restreinte aux administrateurs)

CHAPITRE IV : RÉALISATION ET IMPLÉMENTATION DU SYSTÈME

IV.1 Architecture Logicielle Globale Multi-Surfaces

La solution RÉSIVO adopte une architecture multi-surfaces distribuée, découplant judicieusement le stockage des données, l'administration métier lourde et l'accès nomade des usagers :

• Couche Persistance & Services Cloud : Base de données PostgreSQL / Supabase, assurant le stockage relationnel, la réplication, le stockage objet (photos de pannes) et le canal WebSocket pour la synchronisation temps réel ;
• Couche Client Lourd d'Administration (Desktop) : Application native développée en Python avec le framework PySide6 (Qt6), destinée aux postes administratifs de la cité universitaire ;
• Couche Client Léger Résident (Web) : Portail Web moderne conçu avec le framework Astro, TypeScript et Tailwind CSS, offrant une expérience mobile-first hautement réactive ;
• Couche d'Interconnexion Intelligente (MCP - Model Context Protocol) : Serveur Python interfaçant la base de données avec des outils d'audit et d'inspection programmatique pour la gouvernance automatisée.

IV.2 Environnement Technique et Outillage Logiciel

Le développement a mobilisé une suite d'outils et de bibliothèques éprouvés dans le secteur professionnel :
• Système d'Exploitation : Windows 11 Enterprise (64-bit) ;
• Langages de Programmation : Python 3.12 (Desktop & MCP), TypeScript / JavaScript (Web), PL/pgSQL (Base de données) ;
• Frameworks UI : PySide6 / Qt version 6.7, Tailwind CSS avec tokens graphiques DOU normalisés ;
• Outils de Packaging : PyInstaller 6.22 pour la congélation du binaire Python,

Inno Setup 6.7 pour la génération de l'installeur Windows ;
• Environnement de Test : Pytest (Desktop & MCP), Vitest (Web).

IV.3 Réalisation de la Console Desktop d'Administration

L'interface Desktop constitue le centre nerveux de pilotage de la résidence. Elle offre une navigation latérale escamotable respectant les directives ergonomiques de la Direction des Œuvres Universitaires :

1. Gestion des Pavillons et Chambres :
Une boîte de dialogue dédiée « Residential Blocks » permet d'administrer les blocs (A, B, C...) et leur genre désigné. Lors de la création d'une chambre via « RoomSetupDialog », le sélecteur de bloc renseigne automatiquement le genre qui apparaît verrouillé en lecture seule avec la mention explicite « Automatically inherited from Block ». Le tableau principal affiche des jauges dynamiques d'occupation bicolores (« RoomOccupancyBar ») permettant d'identifier immédiatement les lits vacants.

2. Instruction des Admissions et Filtrage Avancé :
La vue des demandes en attente (« PendingPage ») confronte les données du candidat avec le répertoire officiel importé. La validation déclenche la procédure stockée qui alloue la place de façon atomique sans possibilité de conflit concurrent.

3. Suivi et Clôture des Doléances Techniques :
L'écran de maintenance (« MaintenancePage ») centralise les déclarations transmises par les résidents. L'inspecteur technique peut visualiser les photographies haute résolution attachées à chaque signalement dans une boîte modale dédiée « PhotoViewerDialog » pourvue de fonctions de zoom et d'adaptation plein écran.

IV.4 Réalisation du Portail Web Étudiant

Le portail Web permet à l'étudiant d'interagir avec la résidence depuis tout terminal connecté (ordinateur ou smartphone). L'utilisation d'Astro garantit un

chargement ultra-rapide sans surcoût de rendu JavaScript inutile. Le résident y retrouve son affectation, peut soumettre une réclamation en téléversant un cliché de son sanitaire ou mobilier endommagé, et déposer une demande de permutation de chambre restreinte aux seuls locaux vacants du même genre.

IV.5 Packaging, Distribution et Installeur Windows (Version 0.0.1)

Afin de garantir un déploiement fluide en environnement d'exploitation, l'application Python Desktop a été conditionnée sous forme d'un installeur Windows professionnel créé avec Inno Setup (fichier de sortie RESIVO-Setup-0.0.1.exe) ainsi qu'une version portable autonome zippée (RESIVO-0.0.1-windows-portable.zip). La configuration intègre les polices typographiques, les icônes de marque et les scripts de désinstallation propre.

CHAPITRE V : TESTS, SÉCURITÉ ET VALIDATION DES RÉSULTATS

V.1 Stratégie Globale de Test et Démarche de Validation

La validation d'un système de gestion manipulant des affectations humaines et des données institutionnelles requiert une politique de test intransigeante. Une approche de test multi-niveaux a été déployée, combinant tests unitaires, tests d'intégration avec base simulée (mock) et tests d'invariants réels sur le serveur PostgreSQL.

V.2 Résultats des Suites de Tests Automatisés

L'ensemble des suites de tests s'exécute avec succès et sans régression sur l'ensemble de la base de code :

Tableau 5.1 : Récapitulatif d'exécution des suites de tests automatisés (136 tests validés)

Sous-Système Testé	Outil de Test	Nombre de Tests	Statut	Périmètre de Vérification
Console Desktop (Qt6)	Pytest	29 tests	100% SUCCÈS	Dialogues des blocs, héritage automatique de genre, suppression protégée, jauges d'occupation.
Serveur & Outils MCP	Pytest	97 tests	100% SUCCÈS	Formatage des requêtes d'audit, quotas d'appels, intégrité des identifiants et RPC de lecture.

Portail Web Résident	Vitest	10 tests	100% SUCCÈS	Validation des formulaires d'admission, assainissement des entrées, filtres de permutation.
Total Général	Automatisé	136 tests	CONFORME	Couverture complète des règles de gestion RG-01 à RG-10.

V.3 Validation des Contraintes d'Intégrité de Base de Données

Des scénarios d'attaque et de mauvaise manipulation ont été intentionnellement simulés pour éprouver la solidité du moteur SQL :
• Tentative d'affectation trans-genre : L'insertion d'un étudiant masculin dans une chambre rattachée à un pavillon féminin est rejetée de manière catégorique par l'exception PL/pgSQL 'gender mismatch' ;
• Tentative de sur-capacité : L'assignation d'un résident supplémentaire au-delà de la capacité maximale fixée à la chambre est interceptée ;
• Tentative de suppression d'un pavillon actif : La tentative de suppression d'un bloc abritant des résidents est refusée tant au niveau de l'interface que par la contrainte de clé étrangère RESTRICT / NO ACTION.

CHAPITRE VI : DISCUSSION ET PERSPECTIVES

VI.1 Confrontation des Résultats à la Problématique Initiale

Les résultats tangibles obtenus à l'issue du déploiement expérimental apportent une réponse probante et complète à la question centrale de recherche énoncée dans l'introduction. L'application conjointe des triggers PL/pgSQL et du mécanisme d'héritage de genre au niveau bloc permet d'affirmer que le système résout définitivement les risques d'attribution erronée.

En déportant les contrôles critiques de l'interface utilisateur vers le cœur de la base de données, la sécurité du système ne dépend plus de la vigilance ponctuelle de l'opérateur de saisie, mais de la stricte conformité relationnelle du moteur PostgreSQL.

VI.2 Analyse Comparative et Bénéfices Constatés

L'évaluation qualitative réalisée auprès des personnels d'hébergement met en exergue des bénéfices majeurs :
• Réduction drastique du temps d'affectation : Le traitement complet d'un dossier étudiant passe d'une moyenne de 15 minutes (recherche manuelle de lit vacant, consultation papier) à moins de 30 secondes ;
• Fiabilité absolue des effectifs : Le tableau de bord restitue à chaque instant le nombre exact de lits occupés et vacants, permettant une planification budgétaire optimale pour les approvisionnements de la cité universitaire ;
• Traçabilité infalsifiable des réparations : La transmission de photographies par les résidents élimine les litiges sur l'antériorité des dégradations et responsabilise les corps de métier d'entretien.

VI.3 Limites Identifiées

Certaines limites objectives ont toutefois été identifiées au terme de cette expérimentation :
• Dépendance à la connectivité Internet : Bien que l'application Desktop dispose de mécanismes de tolérance aux coupures brèves, une synchronisation permanente

avec la base cloud requiert un accès réseau stable ;
• Absence de reconnaissance biométrique ou RFID à l'entrée des pavillons, obligeant les agents d'accueil à procéder à des contrôles visuels complémentaires.

VI.4 Perspectives d'Évolution

Dans la continuité de ce travail, plusieurs perspectives prometteuses sont envisageables :

1. Développement d'une application mobile native Android/iOS dédiée aux techniciens de maintenance pour clôturer les interventions par lecture de QR-code apposé sur chaque porte de chambre ;
2. Couplage avec des serrures électroniques connectées ou tourniquets d'accès automatisés pour un contrôle d'accès biométrique ;
3. Algorithmes d'optimisation par affinité académique : intégration d'un module d'intelligence artificielle suggérant le regroupement en chambre des étudiants d'une même filière pour favoriser la réussite universitaire.

CONCLUSION GÉNÉRALE

Le projet RÉSIVO, mené dans le cadre de la préparation au Brevet de Technicien Supérieur en Informatique / Bases de Données, a permis de concevoir, développer et valider une plateforme complète dédiée à la modernisation des résidences universitaires de la Direction des Œuvres Universitaires de Tlemcen.

En abordant le problème sous l'angle rigoureux de l'ingénierie des bases de données relationnelles, ce travail démontre qu'un système d'information solide doit impérativement s'appuyer sur des fondations conceptuelles irréprochables (normalisation 3FN), des verrous d'intégrité déclaratifs et procéduraux (Triggers PL/pgSQL) et une politique de sécurité granulaire par ligne (RLS).

La mise en place de la séparation stricte des blocs par genre, l'héritage automatisé vers les chambres, la gestion photographique des doléances et le déploiement sous forme d'installeur Windows autonome concrétisent l'ensemble des objectifs fixés au début du projet.

Sur le plan professionnel et personnel, l'accomplissement de ce mémoire a constitué une formidable opportunité de mobiliser l'ensemble des compétences acquises au cours des deux années de formation au sein du Centre de formation professionnelle et d'apprentissage des arts traditionnels : administration de bases de données, programmation objet, ergonomie logicielle, protocoles réseau et méthodologie de conduite de projet informatique.

Le système RÉSIVO se présente ainsi comme une solution mature, testée et immédiatement déployable pour hisser la gestion des œuvres universitaires vers les plus hauts standards de modernité, de transparence et d'équité.

BIBLIOGRAPHIE

1. Les Ouvrages et Manuels de Référence

[1] Nanci, D., & Espinasse, B. (2018). Ingénierie des systèmes d'information : Merise deuxième génération (4e éd.). Vuibert.

[2] Gardarin, G. (2014). Bases de données : Objet et relationnel (6e éd.). Eyrolles.

[3] Rigaux, P., & Scholl, M. (2016). Pratique de PostgreSQL : Administration, modélisation et programmation PL/pgSQL. Dunod.

[4] Sommerfield, M. (2020). Rapid GUI Programming with Python and Qt. Prentice Hall.

2. Les Articles et Publications Scientifiques

[5] Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387.

[6] Stonebraker, M., & Rowe, L. A. (1986). The Design of POSTGRES. ACM SIGMOD Record, 15(2), 340-355.

[7] Berners-Lee, T., Fielding, R., & Masinter, L. (2005). Uniform Resource Identifier (URI): Generic Syntax. RFC 3986.

[8] Model Context Protocol Specification. (2024). Open Protocol for AI and External Data Contextualization. Anthropic Technical Report.

3. Les Sites Internet et Documentations Officielles

[9] PostgreSQL Global Development Group. (2026). Documentation Officielle de PostgreSQL 16 & PL/pgSQL. https://www.postgresql.org/docs/

[10] Supabase Inc. (2026). Supabase Database Architecture, Auth & Row Level Security Guides. https://supabase.com/docs/

[11] The Qt Company. (2026). Qt for Python (PySide6) Official API Reference. https://doc.qt.io/qtforpython/

[12] Astro Technology Company. (2026). Astro Web Framework Documentation. https://docs.astro.build/

[13] JRSoftware. (2026). Inno Setup Compiler Documentation. https://jrsoftware.org/isinfo.php

ANNEXES

Plan des Annexes

Annexe I : Script SQL de Définition des Tables Principales (DDL) ..... Page I

Annexe II : Implémentation des Triggers d'Intégrité Métier PL/pgSQL ..... Page II

Annexe III : Procédure Transactionnelle d'Admission d'Étudiant ..... Page III

Annexe IV : Configuration de Compilation de l'Installeur Windows (Inno Setup) ..... Page IV

Annexe I : Script DDL de Création des Tables Relationnelles

■ EXTRAIT SQL DU FICHIER 20261019203000_blocks_and_room_gender.sql

```
CREATE TABLE IF NOT EXISTS blocks (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL UNIQUE,
  gender gender NOT NULL,
  description text,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS rooms (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  block text NOT NULL,
  floor int NOT NULL DEFAULT 0,
  number text NOT NULL,
  capacity int NOT NULL DEFAULT 1 CHECK (capacity > 0),
  gender gender NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (block, number)
);
```

Annexe II : Triggers d'Intégrité Métier (Héritage et Verrouillage)

■ TRIGGER DE SYNCHRONISATION AUTOMATIQUE DU GENRE

```
CREATE OR REPLACE FUNCTION sync_room_gender_from_block()
RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE b_gender gender;
BEGIN
  SELECT gender INTO b_gender FROM blocks WHERE name = NEW.block;
  IF b_gender IS NOT NULL THEN
    NEW.gender := b_gender;
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trigger_sync_room_gender
BEFORE INSERT OR UPDATE ON rooms
FOR EACH ROW EXECUTE FUNCTION sync_room_gender_from_block();
```

Annexe III : Procédure Transactionnelle d'Attribution de Chambre

■ EXTRAIT RPC APPROVE_STUDENT

```
CREATE OR REPLACE FUNCTION approve_student(p_student uuid, p_room uuid)
RETURNS void LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE s students; r rooms; occ int;
BEGIN
  SELECT * INTO s FROM students WHERE id = p_student FOR UPDATE;
  SELECT * INTO r FROM rooms WHERE id = p_room FOR UPDATE;
  IF r.gender <> s.gender THEN
    RAISE EXCEPTION 'Conflit de genre : chambre % et étudiant %', r.gender, s.gender;
  END IF;
  SELECT count(*) INTO occ FROM students WHERE room_id = p_room AND status =
'active';
  IF occ >= r.capacity THEN
    RAISE EXCEPTION 'Chambre complète (% / % places)', occ, r.capacity;
  END IF;
  UPDATE students SET room_id = p_room, status = 'active' WHERE id = p_student;
END;
$$;
```

Annexe IV : Script Inno Setup du Binaire Installeur Windows (v0.0.1)

■ **EXTRAIT** **INSTALLER.ISS**

```
#define MyAppName "RÉSIVO"
#define MyAppVersion "0.0.1"
#define MyAppPublisher "Direction des Œuvres Universitaires · Cité Universitaire · Tlemcen"
[Setup]
AppName={#MyAppName}
AppVersion={#MyAppVersion}
DefaultDirName={autopf}\RESIVO
OutputDir=..\dist\installer
OutputBaseFilename=RESIVO-Setup-0.0.1
Compression=lzma2/ultra64
SolidCompression=yes
```

RÉSUMÉ ET MOTS-CLÉS / ABSTRACT AND KEYWORDS

(À placer sur la page de reliure au dos du mémoire)

Résumé (Français)

Ce mémoire de fin d'études présente la conception et le déploiement de RÉSIVO, un système de gestion intégrée des résidences universitaires élaboré pour la Direction des Œuvres Universitaires de Tlemcen. Face aux limites du traitement manuel et aux risques d'erreurs d'affectation spatiale, une base de données relationnelle PostgreSQL en troisième forme normale a été modélisée. L'intégrité métier est garantie par des triggers PL/pgSQL assurant la création préalable des pavillons (blocs) et l'héritage automatique et inviolable du genre vers les chambres, empêchant toute surcapacité ou assignation non conforme. Le système intègre une console d'administration native en Python/PySide6, un portail Web étudiant en Astro avec téléversement de clichés pour les doléances de maintenance, ainsi qu'une interface programmatique standardisée via le protocole MCP. L'ensemble a été validé par 136 tests unitaires et packagé sous forme d'installeur Windows autonome (version 0.0.1).

Mots-clés : Mots-clés : Base de données relationnelle, PostgreSQL, MERISE, Triggers PL/pgSQL, Row Level Security, Gestion des résidences universitaires, PySide6, Astro, Intégrité référentielle, MCP.

Abstract (English)

This graduation thesis presents the design and deployment of RESIVO, an integrated university residence management system developed for the Directorate of University Works of Tlemcen. Addressing the limitations of manual records and room allocation discrepancies, a third normal form PostgreSQL relational database was engineered. Business integrity is enforced through server-side PL/pgSQL

triggers that require the prior definition of residential blocks and automate gender inheritance to dormitory rooms, preventing capacity overflows and gender mismatches. The architecture combines a native Python/PySide6 desktop administration console, an Astro student web portal supporting photographic maintenance ticket reporting, and a standardized MCP interface for programmatic audit. The system was verified through 136 automated unit tests and distributed as a standalone Windows installer (version 0.0.1).

Keywords : Keywords : Relational Database, PostgreSQL, MERISE, PL/pgSQL Triggers, Row Level Security, University Housing Management, PySide6, Astro, Referential Integrity, Model Context Protocol.